① Problem description

Productive Meeting: n people; person i leaves after a_i talks. Any two present people may talk (repeatedly). Maximize total talks; output count k and the k pairs "i j".

Input: t cases, each: n then a_1 .. a_n.

Example: a = [2, 3] -> k = 2 : (1 2),(1 2)

Forward Specifications Generation F(p)

- Input: t cases; each n then a_1 .. a_n.
- Output: print k, then k pairs i j (i ≠ j).
- Core: use a MAX-HEAP (priority queue) — repeatedly pair the two most sociable, decrement, reinsert.
- Edge: all sociability zero -> k = 0; handle the maximum n / sum a_i efficiently.

② LLM code generation (forward)

```
def max_talks(n, a):
    people = sorted((a[i], i+1)
                    for i in range(n))
    talks = []
    while True:
        people.sort(reverse=True)  # re-sort EVERY talk
        if people[0][0] == 0: break
        x, y = people[0], people[1]  # no guard
        talks.append((x[1], y[1]))
        people[0] = (x[0]-1, x[1])
        people[1] = (y[0]-1, y[1])
    return talks
```

③ Backward Reasoning: Weakest Precondition

Backward Reasoning

L1 people = sorted([...], reverse=True) - plain stmt

Effect: builds (sociability, index) list => final whole-fn spec

L2 while True: - loop ↻ k-induction

Effect: pairs the two most-active, decrements both, until top = 0

L3 people.sort(reverse=True, ...) - plain stmt

Effect: RE-SORTS the whole list every iteration -> O(n log n) / talk

L4 if people[0][0]==0: break - if ⊔ merge

Effect: stops when the most-active person has 0 capacity

L5 first,second = people[0],people[1] - plain stmt

Effect: takes top two; reads people[1] with NO guard (single person)

L6 return results - plain stmt (processed FIRST)

Effect: returns the accumulated (count, pairs) summary per case

initial cont = STOP : "after this point the function returns / ends"

Derived Prompt

- Re-sorts the whole list each round; pairs the two most active, decrements both.
- Accumulates (count, pairs) and returns a per-case summary.
- No heap; reads the 2nd person with no guard.

Final Prompt

```
[ Productive Meeting problem
  statement ... ]

# Make sure the code ALSO
# satisfies:
# - Use a priority queue
#   (max-heap)
# - Output k = 0 when all zero
# - Ensure i != j
# Do NOT do the following:
# - Re-sort & accumulate in a
#   results list
# - Return a per-case summary

Write a COMPLETE Python 3
program ... read stdin, write
stdout. Output ONLY the code.
```

next iteration

pass public test cases?

Yes

No

④ Differences (forward spec ⟷ derived spec)

Missing	max-heap; k = 0 when all zero; i ≠ j
Extra	re-sorts & accumulates in a results list; per-case summary

⑤ Regenerate with Required Prompt

```
import heapq
def max_talks(n, a):
    h = [(-a[i], i+1) for i in range(n)
                      if a[i] > 0]   # only nonzero
    heapq.heapify(h)                 # max-heap
    talks = []
    while len(h) > 1:                # stop at < 2
        x = heapq.heappop(h)
        y = heapq.heappop(h)
        talks.append((x[1], y[1]))
        if -x[0]-1 > 0:
            heapq.heappush(h, (x[0]+1, x[1]))
        if -y[0]-1 > 0:
            heapq.heappush(h, (y[0]+1, y[1]))
    return talks
```